Utility Disruption Notification Module (Stage 1 — Shadow Mode)

Thesis link: §5.4 limitation "Official utility alerts may arrive later than primary-source announcements"; §5.5 future work #3 "real-time official alerts". **Date:** 2026-10-06 (FYP 2, Week 1); **GO-LIVE 2026-10-12 19:25 Asia/Kuching** (owner-approved after the Stage 2 sign-off review, doc 08). **Status: LIVE in production (Fly deploy v120).** Detection, classification, dedup and per-user delivery active for opted-in users; opt-out via `/settings` → "■ Urgent Alerts". First live cycle: 142 deliveries recorded, one dead test account failed gracefully ("Chat not found", logged, non-fatal). Open refinements: incident clustering and restoration threading (Stage 2 items 3–4).

---

1. Problem statement

Users currently learn about water cuts and power outages from the normal news pipeline (15-minute RSS prefetch + evening digests), or not at all. There is no dedicated, fast path for official utility disruption notices — and the FYP 1 evaluation recorded that closed official platforms (Facebook-first agencies) make these alerts arrive late.

2. Source scouting (2026-10-06) — including negative results

Source	Region	Channel	Decision
Sarawak Water (merged KWB/SWB/LAKU, Aug 2025)	Central (Sibu)	**Official Telegram channel** `t.me/swbnews` (6.2K subs, structured notices, several/day)	[OK] Integrated via existing session reader — near-real-time, free
JBALB (Rural Water Supply Dept)	Rural statewide	**Official .gov.my announcement list** — notices dated same-day	[OK] Integrated — direct scrape, polite (1.5 s between detail fetches)
Google News RSS watches	All Sarawak (esp. South/North + power)	Free keyword watches	[OK] Integrated — 15–60 min lag behind official posts
Sarawak Water South/North regions	Kuching/Miri	Facebook pages only	[X] No public Telegram channel found (probed `swsbsouth`, `kwbnews`, `sarawakwater`); FB bridge = optional rss.app subscription (future, ~US$9/mo)
Sarawak Energy	Power	Facebook + X + SEB Cares app	[X] X API is paid (owner-verified) → documented negative result; FB app review impractical for FYP. Covered indirectly via Google News RSS + website media releases. See §2.1 for the 2026-10-13 X-scraping re-check and the SEB Cares web-portal probe
SEB Cares **web portal** (not just app)	Power	`https://sebcares.sarawakenergy.com` (AWS ALB, OutSystems app)	[X] Probed 2026-10-13 with headless-browser network capture: only public pages are Home and Login. Outage alerts come from `POST /SEBCares/screenservices/SEBCares/MainFlow/Home/DataActionGetOutageAlerts`, which returns `Invalid Login` for anonymous callers — alerts are personalized per account + subscribed station, not a public feed. Portal API route rejected; detail in §2.2
Sarawak Water (merged KWB/SWB/LAKU, Aug 2025) — re-confirmed	Central (Sibu)	Official app + Telegram	Already integrated (`t.me/swbnews` via session reader); the app is a shell around the same notices, no extra work needed

2.1 X-scraping landscape re-check (2026-10-13)

Vincent re-asked whether a GitHub tool can scrape the Sarawak Energy X account directly. Current state of that route:

- **Nitter: dead.** Received an X Corp cease-and-desist (2026-08-24); the project is archived and public instances are gone, taking every nitter-based scraper with it.
- **twscrape v0.20.1 (2026-08-25): still works**, but requires your own account's `auth_token` + `ct0` cookies, violates X ToS, risks account bans, and breaks every 2–4 weeks as X rotates its internals. Twikit / Scweet / Tweety are the same account-cookie model with intermittent success.
- **X official API: pay-per-use since Feb 2026**, no free tier — the cost finding from 2026-10-06 still stands.

Verdict: scraping SEB's X profile is *technically* possible via twscrape but operationally unsuitable for a thesis deliverable (ToS violation, burner-account risk, constant maintenance). Recorded here as the negative result; the recommended route is the SEB Cares web-portal API instead.

2.2 SEB Cares portal probe detail (2026-10-13)

Method: Playwright headless browser with full network capture (`scripts/seb_network_probe.js`, kept for reference), plus direct `curl`/`nslookup` checks. Findings:

- Domain: `sebcares.sarawakenergy.com` (`.com` — the `.com.my` variant does not resolve). AWS ALB, ap-southeast-1, OutSystems app.
- Public surface: only `/SEBCares/Home` and `/SEBCares/Login`; the hamburger menu adds no public pages.
- Outage-alert endpoint identified: `POST …/MainFlow/Home/DataActionGetOutageAlerts` — responds `{"exception":…"Invalid Login"}` without a session (verified by direct POST). The data model is per-account, per-subscribed-station (`CustomerServiceLocation`, `PowerAlertType`), so even a burner account would cover only manually subscribed stations.
- Main corporate site (`www.sarawakenergy.com`, public): `media-info/announcements-publications` exists but carries corporate notices (discount extensions, lucky draws; last update Apr 2026), not operational outage notices. `media-info/maintenance-schedules` returns 404.

2.2.1 Session-authenticated probe (2026-10-13, with owner's login)

Vincent supplied a logged-in browser session and asked whether the alert feed could be pulled with his account. Findings from driving his real browser (WebBridge) plus cookie replay:

- Logged-in screens discovered: `/SEBCares/OutageAlertDetail` (tabs: **Unplanned outage** / **Scheduled shutdown**, date-grouped, statewide — e.g. Kuching areas with 09:00–17:00 windows for 12–14 Oct 2026) and `/SEBCares/OutageAlertSearch` (direct navigation → `No Permissions` for customer accounts; likely internal/CS role only).
- Data endpoints (from screen JS): `POST /SEBCares/screenservices/SEBCares/MainFlow/OutageAlertDetail/DataActionGetScheduledList` and `…/DataActionGetUnplannedList`.
- Full call chain reverse-engineered and validated up to the last server check: session `Users` cookie + `X-CSRFToken` header (anonymous token constant embedded in `OutSystems.js`) + per-endpoint `apiVersion` (constant in `SEBCares.MainFlow.OutageAlertDetail.mvc.js`) + body `{"versionInfo":{"moduleVersion","apiVersion"},"viewName","inputParameters"}`. Replay progressed 403 Invalid Login → 400 → 200 with only `No role validation found` remaining.
- **Session expiry observed within the probe window** (~40 min; the portal session idle-expired mid-investigation, confirmed by `$OS_Users$$ClientVars$$SESSION_USER_ID` clearing and the UI reverting to "Log In / Register"). A fresh login would have completed the pull; the technique is sound but the live JSON capture was not completed in-session.
- **Production verdict: not suitable as a bot data source.** The feed is genuinely reachable with a valid customer login, but (a) sessions idle-expire quickly, so automation would require storing the owner's personal SEB credentials in the bot and replaying the login flow — a ToS, security, and 2FA risk; (b) manual cookie refresh is not automation. Suitable only for one-off research pulls. Google News RSS watch remains the shipped route; rss.app FB bridging remains the optional upgrade.

Conclusion: Sarawak Energy publishes no *public* machine-readable operational alert feed. A statewide scheduled/unplanned outage JSON feed does exist behind customer login on the SEB Cares portal (§2.2.1), but its

quick session expiry makes it unfit for unattended bot polling without personal-credential automation — rejected for production, kept as a documented manual/research route. Operationally, the integrated Google News RSS watch remains the shipped route; bridging SEB's public Facebook page via rss.app (~US$9/mo, same mechanism already scoped for Sarawak Water South/North) is the only upgrade path and stays an optional future-work item.

The X API cost and the Meta app-review wall are the recorded "attempt" for thesis §5.5 #3 — "integrate **where feasible**"; the feasible routes are what shipped.

3. Changes made

3.1 Config (`src/core/config.py`)

Variable	Default	Purpose
`UTILITY_ALERT_ENABLED`	`true`	Master switch for the poll job.
`UTILITY_ALERT_SHADOW_MODE`	`true`	**Stage 1: detect + log only, no pushes.** `false` = live delivery.
`UTILITY_ALERT_POLL_MINUTES`	`10`	Poll cadence.
`UTILITY_ALERT_LOOKBACK_HOURS`	`24`	How far back telegram/news sources look.
`UTILITY_ALERT_MAX_PER_HOUR_PER_USER`	`3`	Per-user rate cap.
`UTILITY_ALERT_TELEGRAM_CHANNELS`	`swbnews`	Official channel(s) via existing session reader.
`UTILITY_ALERT_JBALB_URLS`	JBALB announcement list	.gov.my scrape targets.
`UTILITY_ALERT_NEWS_QUERIES`	water disruption / pipe burst / power outage	Google News RSS watches (server-side `when:7d` recency operator).

Quiet hours: alerts reuse the scheduled-push window (12am–6am `Asia/Kuching`) — held until 06:00 (owner decision 2026-10-06). Opt-in: `user_preferences.wants_urgent_alerts` (default on; `/settings` toggle ships with Stage 2).

3.2 Storage

- `utility_alerts` — source, source_ref (unique per source), title, text, utility_type (water/power/other), kind (scheduled/unscheduled/restored/info), canonical locations, raw affected-area snippet, announced_at, window, status. Plain columns → works on Postgres **and** SQLite; idempotent metadata-create migration.
- `utility_alert_delivery` — per-user delivery records (same pattern as `UserArticleDelivery`; no global "already sent" flag).
- Retention: rows older than `DB_RETENTION_DAYS` are purged by the poll cycle.

3.3 Detection + classification (`src/scrapers/utility_alert_reader.py`)

- **Rule-based classifier** (English + Bahasa Malaysia keyword sets, word-boundary matching) — runs with Ollama down (rule #8). kind precedence: restored > scheduled > unscheduled; `"scheduled"` cannot fire inside `"unscheduled"` (word boundaries).
- Location extraction reuses the shared `SARAWAK_LOCATION_ALIASES` taxonomy (rule #4).
- JBALB: list page → matching notice links → detail pages (bodies load via AJAX, so the stored text is title-carried; boilerplate stripped).
- News watch: Google News RSS ranks plain `q=` by relevance (months-old hits), so the fetcher appends `when:7d` server-side + a client-side 48 h age cap, and requires a Sarawak signal ("sarawak" in text or a canonical location) — rejects e.g. Myanmar El Niño outage news.

3.4 Service + wiring

- `src/core/utility_alert_service.py` — `poll_utility_alerts()` (fetch → classify → dedup → insert → log; per-source failures are counted, never fatal), `compute_pending_deliveries()` (location matching, quiet hours, rate cap, per-user delivery skip), `format_utility_alert_html()` (HTML-escaped per rule #7).

- Dedup: exact `source_ref` + title-Jaccard (48 h lookback). Title-only tokens after finding JBALB's AJAX boilerplate made every notice look identical.
- `bot_main.py`: repeating job at `UTILITY_ALERT_POLL_MINUTES` (blocking work in `asyncio.to_thread`, rule #6); sends only when shadow mode is off.
- `/devutility` developer preview (same allow-list as `/devwaze`).

4. Shadow-mode evaluation (2026-10-06, live sources)

Artifact: `benchmark/utility_alerts_shadow_20261006.txt`.

```
poll 1: fetched 16 (telegram 4, jbalb 8, news 4) → classified 10 → new 10
poll 2: new 0, duplicates 10 (idempotent re-poll — nothing double-inserted)
stored: jbalb 8 (7 locations: Betong ×2, Bintulu, Sarikei ×2, Serian, Sibu/Selangau,
Samarahan), telegram 2 (Sibu pipe works, unscheduled notice)
rejected 6: Myanmar El Niño outage (no Sarawak signal), 2 non-notice channel posts,
3 off-topic news items
pending deliveries computed: 155 (53 opted-in users × 3/h cap on a fresh table —
steady state tracks actual new alerts)
```

Dedup/idempotency verified (second poll stores nothing); per-source failure handling verified live when JBALB briefly throttled repeated debug fetches (warnings logged, cycle continued — the politeness delay was added after).

5. Stage 1b — multi-day shadow log protocol (running 2026-10-06 →)

Goal: accumulate several days of `[utility-alert]` logs (ideally spanning at least one 00:00–06:00 Asia/Kuching window, and wet-weather days when JBALB posts more) as the tuning/evidence set before Stage 2 sign-off.

How to run the local bot with capture (from repo root, venv active):

```
powershell -ExecutionPolicy Bypass -File scripts/run_shadow_mode.ps1
# -> console is tee'd to logs/utility_shadow_YYYYMMDD.log (logs/ is gitignored)
```

Git Bash alternative: `python -m src.bot.bot_main 2>&1 | tee -a logs/utility_shadow_$(date +%Y%m%d).log`

Evidence collected: `logs/utility_shadow_*.log` (`[utility-alert]` lines), `/devutility` previews, and `utility_alerts` row counts by source/kind/location.

Stage 2 acceptance criteria (measured from the shadow log):

1. **Precision ≥ 90%** — sample ≥ 20 stored alerts; genuine Sarawak utility notices.
2. **Zero pushes** — only `[shadow] would deliver` lines; no send attempts.
3. **Quiet hours** — alerts announced 00:00–06:00 are held to 06:00 (log timestamps).
4. **Rate cap** — would-deliver per user ≤ 3/h.
5. **Stability** — job ticks every 10 min; errors are rare and non-fatal per source.
6. **No cross-poll duplicates** over the whole period.

An automated check-in on **2026-10-09 (Asia/Kuching)** reads the accumulated log and summarizes it against these criteria as a progress-report entry.

5.1 Durable evidence store (added 2026-10-11, commit `855f2f8`)

The stdout log stream (`flyctl logs`) is **not an archive** — old lines cannot be relied on after days. Shadow evidence is therefore also persisted to the `utility_alert_shadow_log` table (Postgres + SQLite), written by the poll job itself:

- `kind='poll'` — one row per detection cycle with the full summary JSON, timestamped. The unbroken sequence of rows *is* the stability proof (expect `144/day` at the default 10-min cadence); per-cycle duplicate counts prove idempotency over the whole period.
- `kind='would_deliver'` — one row per `(telegram_id, utility_alert_id)` at the **first moment that pair becomes due**, with the alert title/locations as JSON. The unique constraint `(kind, telegram_id, utility_alert_id)` silently skips re-computed pairs, so volume stays ~one row per genuine notification event even though shadow mode never records a real delivery to suppress recomputation.

• *Quiet-hours evidence:* an alert announced 00:00–06:00 only becomes due at 06:00, so its `created_at` ≥ 06:00 demonstrates the hold.

• *Rate-cap evidence:* count rows per user per hour — never above `UTILITY_ALERT_MAX_PER_HOUR_PER_USER`.

Zero pushes is proven by `utility_alert_delivery` staying empty while shadow mode is on.

Gating and safety: writes happen only while `UTILITY_ALERT_SHADOW_MODE=true`, are controlled by `UTILITY_ALERT_SHADOW_LOG` (default on), are best-effort (a failed write can never break the detection cycle), and rows are purged with `DB_RETENTION_DAYS` by the existing alert cleanup.

Review queries for the Stage 2 sign-off:

```
-- Stability: poll ticks per hour (expect 6/hour, no gaps)
SELECT date_trunc('hour', created_at) AS hr, count(*) FROM utility_alert_shadow_log
WHERE kind = 'poll' GROUP BY 1 ORDER BY 1;

-- Quiet hours: first-due time vs alert announcement (hold → created_at ≥ 06:00 local)
SELECT w.created_at, w.payload FROM utility_alert_shadow_log w
WHERE w.kind = 'would_deliver' ORDER BY w.created_at;

-- Rate cap: would-deliver events per user per hour
SELECT telegram_id, date_trunc('hour', created_at) AS hr, count(*)
FROM utility_alert_shadow_log WHERE kind = 'would_deliver'
GROUP BY 1, 2 ORDER BY 3 DESC;
```

On Fly.io the table lives in Supabase, so evidence survives even total loss of the app; `flyctl logs` remains useful only for same-day debugging.

Quick review: `/devshadowlog` (developer allow-list, private chat) renders the same stats as a pass/fail report straight from the table — poll ticks + gap count, would-deliver totals, quiet-hours violations (Kuching local), per-user/hour rate-cap max, and the zero-push check — reviewable from a phone without opening Supabase.

6. Stage 2 checklist (go-live, requires owner approval)

1. Review the Stage 1b shadow log against the acceptance criteria above. [OK] done — doc 08 (2026-10-12).
2. Add `/settings` toggle for `wants_urgent_alerts` (column already exists). [OK] **already live** (verified in code; correction recorded in doc 08).
3. Incident-level clustering for the news watch (multiple outlets report the same burst; reuse Phase 3 semantics) so users get one alert per incident, not per story. ■ refinement, post-go-live.
4. Link "repair completed" follow-ups to their original notice (thread context) for restoration notifications. ■ refinement, post-go-live.
5. Optional: rss.app bridge on Sarawak Water South FB page for near-real-time Kuching coverage (paid). ■ owner decision deferred.
6. Set `UTILITY_ALERT_SHADOW_MODE=false` and redeploy. [OK] **done 2026-10-12** (owner-approved; Fly v120, `UTILITY_ALERT_SHADOW_MODE=false` in `fly.toml`; first live cycle 142 deliveries, one dead test account failed gracefully).

7. Threats to validity / risks

1. Google News watch latency (15–60 min) is out of our control; Telegram/JBALB are near-real-time but cover Central + rural respectively. South/North water and power stay news-mediated until an official channel appears (or rss.app is subscribed).
2. Rule-based classification can miss novel phrasings; shadow logs are the tuning set. LLM enrichment is future work (must degrade gracefully, rule #8).
3. JBALB detail bodies are AJAX-loaded — if their CMS changes, titles still carry location/date, but affected-area lists will be absent until the scraper is adjusted.
4. False-positive alerts erode trust quickly — shadow mode exists exactly to measure precision before users see anything.
5. **Location matching is exact set intersection with no parent/child rollup** (noted at go-live, 2026-10-12): an alert extracted at division granularity ("Betong") does not reach a user who subscribed only to a sub-area ("Saratok"), and vice versa. Deliberately deferred as a documented limitation — the natural fix is a sub-area →

division rollup in `_matching_user_ids`, to ship alongside the post-go-live refinements (clustering, restoration threading).

6. **Go-live stale-notice burst (found and fixed 2026-10-12, same evening):** delivery eligibility keyed on `created_at` (when *our bot* first stored a notice). JBALB backfills days-old notices, so after the shadow→live flip the bot dumped every notice stored in 48 h — including works dated 10-09/10-10 and a 14–16.10 future window — to matched users (366 deliveries, draining at 3/user/h). Fix: eligibility now uses the **event time** — `announced_at` must fall within `[now − 36h, now + 48h]` (`UTILITY_ALERT_DELIVER_PAST_HOURS` / `UTILITY_ALERT_DELIVER_FUTURE_HOURS`); undated rows get a one-shot 24 h storage window. Post-fix dry run: pending collapsed from the stale backlog to the 3 same-day notices. Lesson for the Phase-4 playbook: any backlog-replay path must filter on event time, never storage time.